

Folder server

29 printable files

(file list disabled)

server\.env

```
1 MONGO_URI=mongodb://localhost:27017/ecommerce
2 SECRET_KEY=my-secret-is-here-with-you-so-what-you-gonna-say
3
4 CLOUDINARY_CLOUD_NAME=daucssq7j
5 CLOUDINARY_API_KEY=384892684148627
6 CLOUDINARY_API_SECRET=P5ftPXeG4u4AKNpM631p4cJ-3Xs
7
8 STRIPE_SECRET_KEY=sk_test_51SK1rj7c9l4eXzwynAj2xKf1YIjtKEedFU88ZQiRl22j7ZpAqwVt9pdpnvTMEqIY98TnYx1THuNJ2fg3JEJf6Gjj004HQkasQv
9 STRIPE_PUBLISHABLE_KEY=pk_test_51SK1rj7c9l4eXzwyuRAHQwBpZiRScpUQ9CBaXb8wKtwz1BdKLM4Ycg008o31ATacIsGDbNykQKJLzXP8iKrGF8Qu00XEYLl6tc
10 STRIPE_WEBHOOK_SECRET=whsec_251c51d70679fdbaa2d766dd17dfd47b2cbbfb528d3e63e50a19304cb1eb1fbd
11 CURRENCY=usd
12
13 CLIENT_URL=http://localhost:5173
```

server\config\db.js

```
1 import mongoose from "mongoose";
2 import dotenv from "dotenv";
3 dotenv.config();
4
5 const connectDB = async () => {
6   try {
7     await mongoose.connect(process.env.MONGO_URI, {
8       useNewUrlParser: true,
9       useUnifiedTopology: true,
10     });
11     console.log("MongoDB connected successfully");
12   } catch (error) {
13     console.error("Database connection failed:", error.message);
14     process.exit(1);
15   }
16 };
17
18 export default connectDB;
```

server\controllers\auth.js

```
1  import bcrypt from "bcrypt";
2  import jwt from "jsonwebtoken";
3  import User from "../models/User.js";
4  import dotenv from 'dotenv';
5  dotenv.config()
6
7  export const register = async (req, res) => {
8    try {
9      const { name, email, password, role } = req.body;
10
11      if (!name || !email || !password || !role) {
12        return res
13          .status(400)
14          .json({ success: false, message: "All fields are required" });
15      }
16
17      const existingUser = await User.findOne({ email });
18      if (existingUser)
19        return res
20          .status(400)
21          .json({ success: false, message: "User already exists" });
22
23      const hashedPassword = await bcrypt.hash(password, 10);
24
25      const newUser = await User.create({
26        name, // ✅ matches frontend
27        email,
28        password: hashedPassword,
29        role: role || "customer",
30      });
31
32      const token = jwt.sign(
33        { id: newUser._id, role: newUser.role },
34        process.env.SECRET_KEY,
35        { expiresIn: "7d" }
36      );
37
38      res.status(201).json({
```

```
39     success: true,
40     message: "User registered successfully",
41     token,
42     user: {
43       id: newUser._id,
44       name: newUser.name,
45       email: newUser.email,
46       role: newUser.role,
47     },
48   });
49 } catch (error) {
50   console.error("❌ Register error:", error.message);
51   res.status(500).json({ success: false, message: "Server error" });
52 }
53 };
54
55 export const login = async (req, res) => {
56   try {
57     const { email, password } = req.body;
58
59     if (!email || !password)
60       return res
61         .status(400)
62         .json({ success: false, message: "Email and password are required" });
63
64     const user = await User.findOne({ email }).select("+password");
65     if (!user)
66       return res
67         .status(400)
68         .json({ success: false, message: "Invalid credentials" });
69
70     const isMatch = await bcrypt.compare(password, user.password);
71     if (!isMatch)
72       return res
73         .status(400)
74         .json({ success: false, message: "Invalid credentials" });
75
76     const token = jwt.sign(
77       { id: user._id, role: user.role },
78       process.env.SECRET_KEY,
79       { expiresIn: "7d" }
80     );
```

```
81
82     res.status(200).json({
83       success: true,
84       message: "Login successful",
85       token,
86       user: {
87         id: user._id,
88         name: user.name,
89         email: user.email,
90         role: user.role,
91       },
92     });
93   } catch (error) {
94     console.error("❌ Login error:", error.message);
95     res.status(500).json({ success: false, message: "Server error" });
96   }
97 };
98
99
100 export const logout = async (req, res) => {
101   try {
102     if (req.user && req.user.id) {
103       const user = await User.findById(req.user.id);
104       if (user) {
105         user.lastLogout = new Date();
106         await user.save();
107       }
108     }
109
110
111     res.setHeader("Authorization", "");
112     res.clearCookie("auth_token");
113     res.clearCookie("refresh_token");
114
115     res.status(200).json({
116       success: true,
117       message: "Logout successful",
118     });
119   } catch (error) {
120     console.error("Logout error:", error);
121
122     res.status(200).json({
```

```

123     success: true,
124     message: "Logout completed",
125   });
126 }
127 };
128
129 export const getMe = async (req,res)=>{
130   try {
131     const user = await User.findById(req.user.id);
132
133     if(!user){
134       return res.status(400).json({
135         success:false,
136         message:"User is not found"
137       })
138     };
139
140     res.status(200).json({
141       success:true,
142       user:{
143         id:user._id,
144         name:user.name,
145         // lastName:user.lastName,
146         email:user.email,
147         role:user.role,
148       }
149     })
150
151   } catch (error) {
152     console.error("Get me error:", error);
153     res.status(500).json({
154       success: false,
155       message: "Server error",
156     });
157   }
158 }

```

server\controllers\cartController.js

```

1 // // import Cart from "../models/Cart.js";
2
3 // // export const addToCart = async (req, res) => {

```

```

4 // // try {
5 // //   const userId = req.user._id;
6 // //   const { productId, quantity = 1, price } = req.body;
7
8 // //   if (!productId || !price) {
9 // //     return res
10 // //       .status(400)
11 // //       .json({ success: false, message: "productId and price are required" });
12 // //   }
13
14 // //   let cart = await Cart.findOne({ user: userId });
15 // //   if (!cart) {
16 // //     cart = new Cart({
17 // //       user: userId,
18 // //       items: [{ product: productId, quantity, price }],
19 // //       totalPrice: quantity * price,
20 // //     });
21 // //   } else {
22 // //     const itemIndex = cart.items.findIndex(
23 // //       (item) => item.product.toString() === productId
24 // //     );
25 // //     if (itemIndex > -1) {
26 // //       cart.items[itemIndex].quantity += quantity;
27 // //     } else {
28 // //       cart.items.push({ product: productId, quantity, price });
29 // //     }
30 // //     cart.totalPrice = cart.items.reduce(
31 // //       (total, item) => total + item.quantity * item.price,
32 // //       0
33 // //     );
34 // //   }
35
36 // //   await cart.save();
37 // //   res.status(200).json({ success: true, cart });
38 // // } catch (error) {
39 // //   res.status(500).json({ success: false, message: error.message });
40 // // }
41 // // };
42
43 // // export const getUserCart = async (req, res) => {
44 // //   try {
45 // //     const userId = req.user._id;

```

```
46 // //      const cart = await Cart.findOne({ user: userId }).populate("items.product");
47 // //      if (!cart)
48 // //          return res
49 // //              .status(404)
50 // //              .json({ success: false, message: "Cart not found" });
51 // //      res.status(200).json({ success: true, cart });
52 // //  } catch (error) {
53 // //      res.status(500).json({ success: false, message: error.message });
54 // //  }
55 // // };
56
57 // // export const updateItemQuantity = async (req, res) => {
58 // //     try {
59 // //         const userId = req.user._id;
60 // //         const { productId, quantity } = req.body;
61 // //         const cart = await Cart.findOne({ user: userId });
62 // //         if (!cart)
63 // //             return res
64 // //                 .status(404)
65 // //                 .json({ success: false, message: "Cart not found" });
66
67 // //         const item = cart.items.find((i) => i.product.toString() === productId);
68 // //         if (!item)
69 // //             return res
70 // //                 .status(404)
71 // //                 .json({ success: false, message: "Item not found in cart" });
72
73 // //         item.quantity = quantity;
74 // //         cart.totalPrice = cart.items.reduce(
75 // //             (total, it) => total + it.quantity * it.price,
76 // //             0
77 // //         );
78 // //         await cart.save();
79 // //         res.status(200).json({ success: true, cart });
80 // //     } catch (error) {
81 // //         res.status(500).json({ success: false, message: error.message });
82 // //     }
83 // // };
84
85 // // export const removeCartItem = async (req, res) => {
86 // //     try {
87 // //         const userId = req.user._id;
```

```
88 // //      const { productId } = req.params;
89
90 // //      const cart = await Cart.findOne({ user: userId });
91 // //      if (!cart)
92 // //          return res
93 // //              .status(404)
94 // //              .json({ success: false, message: "Cart not found" });
95
96 // //      cart.items = cart.items.filter(
97 // //          (item) => item.product.toString() !== productId
98 // //      );
99 // //      cart.totalPrice = cart.items.reduce(
100 // //          (total, item) => total + item.quantity * item.price,
101 // //          0
102 // //      );
103
104 // //      await cart.save();
105 // //      res.status(200).json({ success: true, message: "Item removed", cart });
106 // //  } catch (error) {
107 // //      res.status(500).json({ success: false, message: error.message });
108 // //  }
109 // // };
110
111 // import Cart from "../models/Cart.js";
112 // import Product from "../models/Product.js";
113
114 // // Add to cart
115 // export const addToCart = async (req, res) => {
116 //     try {
117 //         const { productId, quantity = 1 } = req.body;
118
119 //         if (!productId) {
120 //             return res.status(400).json({ message: "Product ID is required" });
121 //         }
122
123 //         const product = await Product.findById(productId);
124 //         if (!product) {
125 //             return res.status(404).json({ message: "Product not found" });
126 //         }
127
128 //         // Find user's cart or create one
129 //         let cart = await Cart.findOne({ user: req.user._id });
```

```

130 //      if (!cart) {
131 //          cart = new Cart({ user: req.user._id, items: [] });
132 //      }
133
134 //      // Check if product is already in cart
135 //      const existingItem = cart.items.find(
136 //          (item) => item.product.toString() === productId
137 //      );
138
139 //      if (existingItem) {
140 //          existingItem.quantity += quantity;
141 //      } else {
142 //          cart.items.push({ product: productId, quantity });
143 //      }
144
145 //      await cart.save();
146 //      await cart.populate("items.product"); // populate product details
147
148 //      const total = cart.items.reduce(
149 //          (sum, item) => sum + item.product.new_price * item.quantity,
150 //          0
151 //      );
152
153 //      res
154 //          .status(201)
155 //          .json({ message: "Added to cart", items: cart.items, total });
156 //      } catch (error) {
157 //          console.error("Add to cart error:", error);
158 //          res.status(500).json({ message: "Failed to add to cart" });
159 //      }
160 //  };
161
162 // // Get cart
163 // export const getCart = async (req, res) => {
164 //     try {
165 //         const cart = await Cart.findOne({ user: req.user._id }).populate(
166 //             "items.product"
167 //         );
168 //         if (!cart) {
169 //             return res.status(200).json({ items: [], total: 0 });
170 //         }
171 //         const total = cart.items.reduce(

```

```

172 //      (sum, item) => sum + item.product.new_price * item.quantity,
173 //      0
174 //    });
175 //    res.status(200).json({ items: cart.items, total });
176 //  } catch (error) {
177 //    console.error("Get cart error:", error);
178 //    res.status(500).json({ message: "Failed to fetch cart" });
179 //  }
180 // };
181
182 // // 📄 Update quantity
183 // export const updateCartItem = async (req, res) => {
184 //   try {
185 //     const { itemId } = req.params;
186 //     const { quantity } = req.body;
187
188 //     const cart = await Cart.findOne({ user: req.user.id });
189 //     if (!cart) return res.status(404).json({ message: "Cart not found" });
190
191 //     const item = cart.items.id(itemId);
192 //     if (!item) return res.status(404).json({ message: "Item not found" });
193
194 //     item.quantity = quantity;
195 //     await cart.save();
196 //     await cart.populate("items.product");
197
198 //     const total = cart.items.reduce(
199 //       (sum, item) => sum + item.product.new_price * item.quantity,
200 //       0
201 //     );
202
203 //     res.json({ message: "Cart updated", items: cart.items, total });
204 //   } catch (error) {
205 //     console.error("Error updating cart:", error);
206 //     res.status(500).json({ message: "Failed to update cart" });
207 //   }
208 // };
209
210 // // ❌ Remove item
211 // export const removeFromCart = async (req, res) => {
212 //   try {
213 //     const { itemId } = req.params;

```

```
214 //      const cart = await Cart.findOne({ user: req.user.id });
215 //      if (!cart) return res.status(404).json({ message: "Cart not found" });
216
217
218 //      cart.items = cart.items.filter((item) => item.id !== itemId);
219 //      await cart.save();
220 //      await cart.populate("items.product");
221
222 //      const total = cart.items.reduce(
223 //          (sum, item) => sum + item.product.new_price * item.quantity,
224 //          0
225 //      );
226
227 //      res.json({ message: "Item removed", items: cart.items, total });
228 //  } catch (error) {
229 //      console.error("Error removing item:", error);
230 //      res.status(500).json({ message: "Failed to remove item" });
231 //  }
232 // };
233
234 // // 🧹 Clear entire cart
235 // export const clearCart = async (req, res) => {
236 //   try {
237 //     await Cart.findOneAndUpdate({ user: req.user.id }, { items: [] });
238 //     res.json({ message: "Cart cleared", items: [], total: 0 });
239 //   } catch (error) {
240 //     console.error("Error clearing cart:", error);
241 //     res.status(500).json({ message: "Failed to clear cart" });
242 //   }
243 // };
244
245 import Cart from "../models/Cart.js";
246 import Product from "../models/Product.js";
247
248 // Add to cart
249 export const addToCart = async (req, res) => {
250   try {
251     const { productId, quantity = 1 } = req.body;
252
253     if (!productId) {
254       return res.status(400).json({ message: "Product ID is required" });
255     }
256   }
257 }
```

```
256 const product = await Product.findById(productId);
257 if (!product) {
258   return res.status(404).json({ message: "Product not found" });
259 }
260
261 // Find user's cart or create one
262 let cart = await Cart.findOne({ user: req.user._id });
263 if (!cart) {
264   cart = new Cart({ user: req.user._id, items: [] });
265 }
266
267 // Check if product is already in cart
268 const existingItem = cart.items.find(
269   (item) => item.product.toString() === productId
270 );
271
272 if (existingItem) {
273   existingItem.quantity += quantity;
274 } else {
275   cart.items.push({ product: productId, quantity });
276 }
277
278 await cart.save();
279 await cart.populate("items.product"); // populate product details
280
281 const total = cart.items.reduce(
282   (sum, item) =>
283     sum + (item.product.new_price || item.product.price) * item.quantity,
284   0
285 );
286
287 res.status(201).json({
288   message: "Added to cart",
289   items: cart.items,
290   total: parseFloat(total.toFixed(2)),
291 });
292 } catch (error) {
293   console.error("Add to cart error:", error);
294   res.status(500).json({ message: "Failed to add to cart" });
295 }
296 }
297 };
```

```
298 // Get cart
299
300 export const getCart = async (req, res) => {
301   try {
302     const cart = await Cart.findOne({ user: req.user._id }).populate(
303       "items.product"
304     );
305
306     if (!cart || !cart.items.length) {
307       return res.status(200).json({ items: [], total: 0 });
308     }
309
310     const total = cart.items.reduce(
311       (sum, item) =>
312         sum +
313         (item.product.new_price || item.product.price || 0) * item.quantity,
314       0
315     );
316
317     res.status(200).json({
318       items: cart.items,
319       total: parseFloat(total.toFixed(2)),
320     });
321   } catch (error) {
322     console.error("Get cart error:", error);
323     res.status(500).json({ message: "Failed to fetch cart" });
324   }
325 };
326
327 // Update quantity
328 export const updateCartItem = async (req, res) => {
329   try {
330     const { itemId } = req.params;
331     const { quantity } = req.body;
332
333     if (quantity < 1) {
334       return res.status(400).json({ message: "Quantity must be at least 1" });
335     }
336
337     const cart = await Cart.findOne({ user: req.user._id });
338     if (!cart) return res.status(404).json({ message: "Cart not found" });
339
```

```
340 // Find the item by _id (MongoDB's default ID field)
341 const item = cart.items.find((item) => item._id.toString() === itemId);
342 if (!item)
343     return res.status(404).json({ message: "Item not found in cart" });
344
345 item.quantity = quantity;
346 await cart.save();
347 await cart.populate("items.product");
348
349 const total = cart.items.reduce(
350     (sum, item) =>
351         sum +
352         (item.product.new_price || item.product.price || 0) * item.quantity,
353     0
354 );
355
356 res.json({
357     message: "Cart updated",
358     items: cart.items,
359     total: parseFloat(total.toFixed(2)),
360 });
361 } catch (error) {
362     console.error("Error updating cart:", error);
363     res.status(500).json({ message: "Failed to update cart" });
364 }
365 };
366
367 // Remove item
368 export const removeFromCart = async (req, res) => {
369     try {
370         const { itemId } = req.params;
371
372         const cart = await Cart.findOne({ user: req.user._id });
373         if (!cart) return res.status(404).json({ message: "Cart not found" });
374
375         // Filter by _id instead of id
376         cart.items = cart.items.filter((item) => item._id.toString() !== itemId);
377         await cart.save();
378         await cart.populate("items.product");
379
380         const total = cart.items.reduce(
381             (sum, item) =>
```

```

382         sum +
383         (item.product.new_price || item.product.price || 0) * item.quantity,
384         0
385     );
386
387     res.json({
388         message: "Item removed",
389         items: cart.items,
390         total: parseFloat(total.toFixed(2)),
391     });
392 } catch (error) {
393     console.error("Error removing item:", error);
394     res.status(500).json({ message: "Failed to remove item" });
395 }
396 };
397
398 // Clear entire cart
399 export const clearCart = async (req, res) => {
400     try {
401         const cart = await Cart.findOne({ user: req.user._id });
402         if (!cart) return res.status(404).json({ message: "Cart not found" });
403
404         cart.items = [];
405         await cart.save();
406
407         res.json({
408             message: "Cart cleared",
409             items: [],
410             total: 0,
411         });
412     } catch (error) {
413         console.error("Error clearing cart:", error);
414         res.status(500).json({ message: "Failed to clear cart" });
415     }
416 };
417

```

server\controllers\categoryController.js

```

1 import Category from "../models/Category.js";
2 import Product from "../models/Product.js";
3

```

```
4 export const createCategory = async (req, res) => {
5   try {
6     const { name, description } = req.body;
7
8     if (!name) {
9       return res.status(400).json({
10         success: false,
11         message: "name is required",
12       });
13     }
14
15     if (req.user.role !== "admin") {
16       return res.status(403).json({
17         success: false,
18         message: "Access denied. Only admin can create categories.",
19       });
20     }
21
22     const existingCategory = await Category.findOne({ name });
23     if (existingCategory) {
24       return res.status(400).json({
25         success: false,
26         message: "Category already exists",
27       });
28     }
29
30     const category = await Category.create({
31       name,
32       description,
33     });
34
35     res.status(201).json({
36       success: true,
37       message: "Category created successfully",
38       category,
39     });
40   } catch (error) {
41     res.status(500).json({
42       success: false,
43       message: error.message,
44     });
45   }
```

```
46 };
47
48 export const getAllCategories = async (req, res) => {
49   try {
50     const categories = await Category.find();
51     if (!categories || categories.length === 0) {
52       return res.status(404).json({
53         success: false,
54         message: "No categories found",
55       });
56     }
57
58     res.status(200).json({
59       count: categories.length,
60       success: true,
61       categories,
62     });
63   } catch (error) {
64     res.status(500).json({
65       success: false,
66       message: error.message,
67     });
68   }
69 };
70
71 export const updateCategory = async (req, res) => {
72   try {
73     const { name, description } = req.body;
74
75     if (req.user.role !== "admin") {
76       return res.status(400).json({
77         success: false,
78         message: "Access denied. Only admin can update categories.",
79       });
80     }
81
82     const category = await Category.findById(req.params.id);
83     if (!category) {
84       return res.status(400).json({
85         success: false,
86         message: "category not found",
87       });
88     }
89   }
90 }
```

```
88     }
89
90     const updatedCategory = await Category.findByIdAndUpdate(
91       req.params.id,
92       {
93         name,
94         description,
95       },
96       {
97         new: true,
98         runValidators: true,
99       }
100     );
101
102     res.status(200).json({
103       success: true,
104       message: "Category updated successfully",
105       category: updatedCategory,
106     });
107   } catch (error) {
108     res.status(500).json({
109       success: false,
110       message: error.message,
111     });
112   }
113 };
114
115 export const deleteCategory = async (req, res) => {
116   try {
117     if (req.user.role !== "admin") {
118       return res.status(400).json({
119         success: false,
120         message: "Access denied. Only admin can delete categories.",
121       });
122     }
123
124     const category = await Category.findById(req.params.id);
125     if (!category) {
126       return res.status(404).json({
127         success: false,
128         message: "Category not found",
129       });
130     }
131   }
132 }
```

```

130     }
131
132     await Category.findByIdAndDelete(req.params.id);
133
134     res.status(200).json({
135         success: true,
136         message: "Category deleted successfully!",
137     });
138 } catch (error) {
139     res.status(500).json({
140         success: false,
141         message: error.message,
142     });
143 }
144 };
145

```

server\controllers\orderController.js

```

1  import Order from "../models/Order.js";
2  import Cart from "../models/Cart.js";
3
4  export const createOrder = async (req, res) => {
5      try {
6          const userId = req.user._id;
7          const { shippingAddress } = req.body;
8
9          console.log("Creating order for user:", userId);
10         console.log("Shipping address:", shippingAddress);
11
12         // Validate shipping address
13         if (
14             !shippingAddress ||
15             !shippingAddress.address ||
16             !shippingAddress.city ||
17             !shippingAddress.country
18         ) {
19             return res.status(400).json({
20                 success: false,
21                 message: "Shipping address with address, city, and country is required",
22             });
23         }

```

```
24 const cart = await Cart.findOne({ user: userId }).populate("items.product");
25 console.log("Cart found:", cart);
26
27
28 if (!cart || cart.items.length === 0) {
29     return res.status(400).json({
30         success: false,
31         message: "Cart is empty or not found",
32     });
33 }
34
35 // Debug: Check cart items structure
36 console.log("Cart items:", cart.items);
37 cart.items.forEach((item, index) => {
38     console.log(`Item ${index}:`, {
39         product: item.product,
40         quantity: item.quantity,
41         productPrice: item.product?.price,
42         productNewPrice: item.product?.new_price,
43     });
44 });
45
46 // Calculate total price and prepare order items
47 const orderItems = cart.items.map((item) => {
48     // Get the price from the product - handle both price and new_price fields
49     const price = item.product?.new_price || item.product?.price;
50
51     if (!price) {
52         throw new Error(
53             `Price not found for product: ${
54                 item.product?.name || "Unknown product"
55             }`
56         );
57     }
58
59     if (isNaN(price)) {
60         throw new Error(
61             `Invalid price for product: ${
62                 item.product?.name || "Unknown product"
63             }`
64         );
65     }
66 }
```

```

66     return {
67         product: item.product._id,
68         quantity: item.quantity,
69         price: parseFloat(price), // Ensure it's a number
70     };
71 });
72
73
74 const totalPrice = orderItems.reduce(
75     (acc, item) => acc + item.price * item.quantity,
76     0
77 );
78
79 console.log("Order items:", orderItems);
80 console.log("Total price:", totalPrice);
81
82 // Create the order
83 const order = await Order.create({
84     user: userId,
85     orderItems,
86     shippingAddress: {
87         address: shippingAddress.address.trim(),
88         city: shippingAddress.city.trim(),
89         postalCode: shippingAddress.postalCode?.trim() || "",
90         country: shippingAddress.country.trim(),
91     },
92     totalPrice: parseFloat(totalPrice.toFixed(2)),
93     paymentMethod: shippingAddress.paymentMethod || "Card",
94 });
95
96 console.log("Order created successfully:", order._id);
97
98 // Clear the cart after successful order
99 await Cart.deleteOne({ user: userId });
100
101 res.status(201).json({
102     success: true,
103     message: "Order placed successfully",
104     order,
105 });
106 } catch (error) {
107     console.error("Order creation error:", error);

```

```
108     res.status(500).json({
109       success: false,
110       message: error.message,
111     });
112   }
113 };
114
115 // ... rest of your controller functions remain the same
116 export const getUserOrders = async (req, res) => {
117   try {
118     const userId = req.user._id;
119     const orders = await Order.find({ user: userId })
120       .populate("orderItems.product")
121       .sort({ createdAt: -1 });
122
123     res.status(200).json({ success: true, orders });
124   } catch (error) {
125     res.status(500).json({ success: false, message: error.message });
126   }
127 };
128
129 export const updateOrderStatus = async (req, res) => {
130   try {
131     const { status } = req.body;
132     const order = await Order.findById(req.params.id);
133     if (!order)
134       return res
135         .status(404)
136         .json({ success: false, message: "Order not found" });
137
138     order.status = status;
139     await order.save();
140
141     res.status(200).json({
142       success: true,
143       message: `Order status updated to ${status}`,
144       order,
145     });
146   } catch (error) {
147     res.status(500).json({ success: false, message: error.message });
148   }
149 };
```

```

150
151 export const deleteOrder = async (req, res) => {
152   try {
153     const order = await Order.findById(req.params.id);
154     if (!order)
155       return res
156         .status(404)
157         .json({ success: false, message: "Order not found" });
158
159     await order.deleteOne();
160     res
161       .status(200)
162       .json({ success: true, message: "Order deleted successfully" });
163   } catch (error) {
164     res.status(500).json({ success: false, message: error.message });
165   }
166 };
167

```

server\controllers\paymentController.js

```

1  import Stripe from "stripe";
2  import Order from "../models/Order.js";
3  import Cart from "../models/Cart.js";
4  import dotenv from "dotenv";
5  dotenv.config();
6
7  const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
8
9  export const createPaymentIntent = async (req, res) => {
10   try {
11     const userId = req.user?.id || req.user?._id;
12     const { orderId } = req.body;
13
14     if (!userId && !orderId) {
15       return res
16         .status(400)
17         .json({ success: false, message: "userId or orderId required" });
18     }
19
20     let amountCents;
21     let order;

```

```
22   if (orderId) {
23     order = await Order.findById(orderId).populate("orderItems.product");
24     if (!order)
25       return res
26         .status(404)
27         .json({ success: false, message: "Order not found" });
28     amountCents = Math.round(order.totalPrice * 100);
29   } else {
30     const cart = await Cart.findOne({ user: userId }).populate(
31       "items.product"
32     );
33     if (!cart || !cart.items.length)
34       return res
35         .status(400)
36         .json({ success: false, message: "Cart not found or empty" });
37     amountCents = Math.round(cart.totalPrice * 100);
38   }
39
40
41   const paymentIntent = await stripe.paymentIntents.create({
42     amount: amountCents,
43     currency: process.env.CURRENCY || "usd",
44     metadata: { orderId: orderId || `fromCart:${userId}` },
45     automatic_payment_methods: { enabled: true },
46   });
47
48   res
49     .status(200)
50     .json({ success: true, clientSecret: paymentIntent.client_secret });
51 } catch (err) {
52   console.error("createPaymentIntent:", err);
53   res.status(500).json({ success: false, message: err.message });
54 }
55 };
56
57 export const markOrderAsPaid = async (req, res) => {
58   try {
59     const order = await Order.findById(req.params.id);
60     if (!order)
61       return res
62         .status(404)
63         .json({ success: false, message: "Order not found" });
```

```

64     order.isPaid = true;
65     order.paidAt = Date.now();
66     order.paymentMethod = req.body.paymentMethod || "Card";
67
68
69     order.paymentResult = {
70       id: req.body.paymentId || "SIMULATED_TXN_" + Date.now(),
71       status: "Completed",
72       update_time: new Date().toISOString(),
73       email_address: req.body.email || "user@example.com",
74     };
75
76     await order.save();
77     res
78       .status(200)
79       .json({
80         success: true,
81         message: "Payment confirmed successfully",
82         order,
83       });
84   } catch (error) {
85     res.status(500).json({ success: false, message: error.message });
86   }
87 };
88

```

server\controllers\productController.js

```

1  import { v2 as cloudinary } from "cloudinary";
2  import Product from "../models/Product.js";
3  import Category from "../models/Category.js";
4
5  export const createProduct = async (req, res) => {
6    try {
7      const { name, description, price, category, stock } = req.body;
8
9      if (!name || !description || !price || !category || !stock) {
10         return res.status(400).json({
11           success: false,
12           message: "All fields are required",
13         });
14       }

```

```
15 // 🔍 Check if category is a valid ObjectId or a name
16 let categoryId = category;
17
18
19 // If it's a string like "laptops", find it by name
20 if (typeof category === "string" && !category.match(/^[0-9a-fA-F]{24}$/)) {
21   const foundCategory = await Category.findOne({
22     name: { $regex: category, $options: "i" },
23   });
24
25   if (!foundCategory) {
26     return res.status(400).json({
27       success: false,
28       message: `Invalid category name: ${category}`,
29     });
30   }
31
32   categoryId = foundCategory._id; // ✅ use ObjectId
33 }
34
35 const images =
36   req.files?.length > 0
37     ? req.files.map((file) => ({
38       url: file.path,
39       filename: file.filename,
40     }))
41     : [];
42
43 const productData = {
44   name,
45   description,
46   price,
47   category: categoryId, // ✅ use resolved ObjectId
48   stock,
49 };
50
51 if (images.length > 0) productData.image = images;
52
53 const product = await Product.create(productData);
54
55 res.status(201).json({
56   success: true,
```

```
57     message: "✅ Product created successfully",
58     product,
59   });
60 } catch (error) {
61   console.error("❌ Error creating product:", error.message);
62   res.status(400).json({
63     success: false,
64     message: error.message,
65   });
66 }
67 };
68
69
70 export const getAllProducts = async (req, res) => {
71   try {
72     const { category, keyword } = req.query;
73     let filter = {};
74
75     if (category) {
76       const cat = await Category.findOne({
77         name: { $regex: category, $options: "i" },
78       });
79       if (cat) filter.category = cat._id;
80     }
81
82     if (keyword) {
83       filter.name = { $regex: keyword, $options: "i" };
84     }
85
86     const products = await Product.find(filter).populate("category", "name");
87     res.json({ count: products.length, products });
88   } catch (err) {
89     res.status(500).json({ success: false, message: err.message });
90   }
91 };
92
93 export const getProductById = async (req, res) => {
94   try {
95     const product = await Product.findById(req.params.id).populate(
96       "category",
97       "name"
98     );
99   }
```

```
99   if (!product)
100     return res
101     .status(404)
102     .json({ success: false, message: "Product not found" });
103   res.status(200).json({ success: true, product });
104 } catch (error) {
105   res.status(400).json({ success: false, message: error.message });
106 }
107 };
108
109 export const updateProduct = async (req, res) => {
110   try {
111     const { name, description, price, category, stock } = req.body;
112
113     const product = await Product.findById(req.params.id);
114     if (!product)
115       return res
116       .status(404)
117       .json({ success: false, message: "Product not found" });
118
119     if (req.user.role !== "admin")
120       return res
121       .status(403)
122       .json({ success: false, message: "Not authorized" });
123
124
125     if (name) product.name = name;
126     if (description) product.description = description;
127     if (price) product.price = price;
128     if (category) product.category = category;
129     if (stock !== undefined) product.stock = stock;
130
131
132     if (req.files && req.files.length > 0) {
133       for (let img of product.image) {
134         await cloudinary.uploader.destroy(img.filename);
135       }
136
137       product.image = req.files.map((file) => ({
138         url: file.path,
139         filename: file.filename,
140       }));
```

```

141     }
142
143     await product.save();
144
145     res.status(200).json({
146         success: true,
147         message: "Product updated successfully",
148         product,
149     });
150 } catch (error) {
151     res.status(400).json({ success: false, message: error.message });
152 }
153 };
154
155
156 export const deleteProduct = async (req, res) => {
157     try {
158         const product = await Product.findById(req.params.id);
159         if (!product)
160             return res
161                 .status(404)
162                 .json({ success: false, message: "Product not found" });
163
164         if (req.user.role !== "admin")
165             return res
166                 .status(403)
167                 .json({ success: false, message: "Not authorized to delete products" });
168
169         await Product.findByIdAndDelete(req.params.id);
170         res
171             .status(200)
172             .json({ success: true, message: "Product deleted successfully" });
173     } catch (error) {
174         res.status(400).json({ success: false, message: error.message });
175     }
176 };
177

```

server\controllers\ratingController.js

```

1 // controllers/ratingController.js
2 import Product from "../models/Product.js";

```

```
3
4 // Add a review/rating
5 export const addReview = async (req, res) => {
6   try {
7     const { productId } = req.params;
8     const { rating, comment } = req.body;
9     const userId = req.user._id;
10    const userName = req.user.name || req.user.username;
11
12    // Validate rating
13    if (!rating || rating < 1 || rating > 5) {
14      return res.status(400).json({
15        success: false,
16        message: "Rating must be between 1 and 5",
17      });
18    }
19
20    if (!comment || comment.trim().length === 0) {
21      return res.status(400).json({
22        success: false,
23        message: "Comment is required",
24      });
25    }
26
27    const product = await Product.findById(productId);
28    if (!product) {
29      return res.status(404).json({
30        success: false,
31        message: "Product not found",
32      });
33    }
34
35    // Check if user already reviewed this product
36    const alreadyReviewed = product.reviews.find(
37      (review) => review.user.toString() === userId.toString()
38    );
39
40    if (alreadyReviewed) {
41      return res.status(400).json({
42        success: false,
43        message: "You have already reviewed this product",
44      });
45    }
46  }
47 }
```

```
45     }
46
47     // Add review
48     const review = {
49       user: userId,
50       userName,
51       rating: Number(rating),
52       comment: comment.trim(),
53     };
54
55     product.reviews.push(review);
56
57     // Calculate average rating
58     product.rating =
59       product.reviews.reduce((acc, item) => item.rating + acc, 0) /
60       product.reviews.length;
61     product.numReviews = product.reviews.length;
62
63     await product.save();
64
65     res.status(201).json({
66       success: true,
67       message: "Review added successfully",
68       review,
69       productRating: product.rating,
70       numReviews: product.numReviews,
71     });
72   } catch (error) {
73     console.error("Add review error:", error);
74     res.status(500).json({
75       success: false,
76       message: error.message,
77     });
78   }
79 };
80
81 // Get product reviews
82 export const getProductReviews = async (req, res) => {
83   try {
84     const { productId } = req.params;
85
86     const product = await Product.findById(productId)
```

```
87     .populate("reviews.user", "name username")
88     .select("reviews rating numReviews");
89
90     if (!product) {
91         return res.status(404).json({
92             success: false,
93             message: "Product not found",
94         });
95     }
96
97     res.status(200).json({
98         success: true,
99         reviews: product.reviews,
100         rating: product.rating,
101         numReviews: product.numReviews,
102     });
103 } catch (error) {
104     console.error("Get reviews error:", error);
105     res.status(500).json({
106         success: false,
107         message: error.message,
108     });
109 }
110 };
111
112 // Update a review
113 export const updateReview = async (req, res) => {
114     try {
115         const { productId, reviewId } = req.params;
116         const { rating, comment } = req.body;
117         const userId = req.user._id;
118
119         const product = await Product.findById(productId);
120         if (!product) {
121             return res.status(404).json({
122                 success: false,
123                 message: "Product not found",
124             });
125         }
126
127         const review = product.reviews.id(reviewId);
128         if (!review) {
```

```
129     return res.status(404).json({
130       success: false,
131       message: "Review not found",
132     });
133   }
134
135   // Check if user owns the review
136   if (review.user.toString() !== userId.toString()) {
137     return res.status(403).json({
138       success: false,
139       message: "Not authorized to update this review",
140     });
141   }
142
143   // Update review
144   if (rating) review.rating = Number(rating);
145   if (comment) review.comment = comment.trim();
146   review.updatedAt = new Date();
147
148   // Recalculate average rating
149   product.rating =
150     product.reviews.reduce((acc, item) => item.rating + acc, 0) /
151     product.reviews.length;
152
153   await product.save();
154
155   res.status(200).json({
156     success: true,
157     message: "Review updated successfully",
158     review,
159     productRating: product.rating,
160   });
161 } catch (error) {
162   console.error("Update review error:", error);
163   res.status(500).json({
164     success: false,
165     message: error.message,
166   });
167 }
168 };
169
170 // Delete a review
```

```
171 export const deleteReview = async (req, res) => {
172   try {
173     const { productId, reviewId } = req.params;
174     const userId = req.user._id;
175
176     const product = await Product.findById(productId);
177     if (!product) {
178       return res.status(404).json({
179         success: false,
180         message: "Product not found",
181       });
182     }
183
184     const review = product.reviews.id(reviewId);
185     if (!review) {
186       return res.status(404).json({
187         success: false,
188         message: "Review not found",
189       });
190     }
191
192     // Check if user owns the review or is admin
193     if (
194       review.user.toString() !== userId.toString() &&
195       req.user.role !== "admin"
196     ) {
197       return res.status(403).json({
198         success: false,
199         message: "Not authorized to delete this review",
200       });
201     }
202
203     // Remove review
204     product.reviews.pull(reviewId);
205
206     // Recalculate average rating
207     if (product.reviews.length > 0) {
208       product.rating =
209         product.reviews.reduce((acc, item) => item.rating + acc, 0) /
210         product.reviews.length;
211       product.numReviews = product.reviews.length;
212     } else {
```

```

213     product.rating = 0;
214     product.numReviews = 0;
215 }
216
217 await product.save();
218
219 res.status(200).json({
220     success: true,
221     message: "Review deleted successfully",
222     productRating: product.rating,
223     numReviews: product.numReviews,
224 });
225 } catch (error) {
226     console.error("Delete review error:", error);
227     res.status(500).json({
228         success: false,
229         message: error.message,
230     });
231 }
232 };
233

```

server\controllers\reviewController.js

```

1 import Product from "../models/Product.js";
2
3 // POST /api/products/:id/reviews
4 export const createReview = async (req, res) => {
5     try {
6         const { name, rating, comment } = req.body;
7         const product = await Product.findById(req.params.id);
8
9         if (!product) return res.status(404).json({ message: "Product not found" });
10
11         const review = { name, rating, comment, date: new Date() };
12
13         product.reviews.push(review);
14         await product.save();
15
16         res.status(201).json(review);
17     } catch (err) {
18         res.status(500).json({ message: err.message });
19     }
20 }

```

```

19 }
20 };
21
22 // GET /api/products/:id/reviews
23 export const getProductReviews = async (req, res) => {
24   try {
25     const product = await Product.findById(req.params.id).select("reviews");
26     if (!product) return res.status(404).json({ message: "Product not found" });
27
28     res.json(product.reviews);
29   } catch (err) {
30     res.status(500).json({ message: err.message });
31   }
32 };
33
34 export const deleteReview = async (req, res) => {
35   try {
36     const { id, reviewId } = req.params;
37     const product = await Product.findById(id);
38
39     if (!product) return res.status(404).json({ message: "Product not found" });
40
41     product.reviews = product.reviews.filter(
42       (r) => r._id.toString() !== reviewId.toString()
43     );
44
45     await product.save();
46     res.status(200).json({ message: "Review deleted successfully" });
47   } catch (error) {
48     res.status(500).json({ message: error.message });
49   }
50 };
51

```

server\controllers\webhookController.js

```

1 import Stripe from "stripe";
2 import mongoose from "mongoose";
3 import Order from "../models/Order.js";
4 import Product from "../models/Product.js";
5
6 const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);

```

```
7
8 export const stripeWebhook = async (req, res) => {
9   const sig = req.headers["stripe-signature"];
10  const webhookSecret = process.env.STRIPE_WEBHOOK_SECRET;
11
12  let event;
13  try {
14    event = stripe.webhooks.constructEvent(req.rawBody, sig, webhookSecret);
15  } catch (err) {
16    console.error("Webhook signature verification failed:", err.message);
17    return res.status(400).send(`Webhook Error: ${err.message}`);
18  }
19
20  try {
21    if (event.type === "payment_intent.succeeded") {
22      const paymentIntent = event.data.object;
23      const metadataOrderId = paymentIntent.metadata?.orderId;
24      let orderId = metadataOrderId;
25
26      if (metadataOrderId?.startsWith("fromCart:")) {
27        console.log("Payment for cart (no orderId):", metadataOrderId);
28      }
29
30      if (orderId) {
31        const session = await mongoose.startSession();
32        try {
33          session.startTransaction();
34
35          const order = await Order.findById(orderId).session(session);
36          if (!order) {
37            console.warn("Order not found for orderId:", orderId);
38            await session.commitTransaction();
39          } else if (!order.isPaid) {
40            order.isPaid = true;
41            order.paidAt = new Date();
42            order.paymentMethod = "Card";
43            order.paymentResult = {
44              id: paymentIntent.id,
45              status: paymentIntent.status,
46              update_time: new Date().toISOString(),
47              email_address: paymentIntent.receipt_email || "",
48            };

```

```

49     for (const item of order.orderItems) {
50         const updateResult = await Product.updateOne(
51             { _id: item.product, stock: { $gte: item.quantity } },
52             { $inc: { stock: -item.quantity } }
53         ).session(session);
54
55         if (updateResult.matchedCount === 0) {
56             throw new Error(
57                 `Insufficient stock for product ${item.product}`
58             );
59         }
60     }
61 }
62
63 await order.save({ session });
64 await session.commitTransaction();
65 console.log(`Order ${orderId} marked paid and stock updated.`);
66 } else {
67     await session.commitTransaction();
68     console.log(`Order ${orderId} already marked as paid.`);
69 }
70 } catch (txErr) {
71     await session.abortTransaction();
72     console.error("Transaction error in webhook:", txErr);
73 } finally {
74     session.endSession();
75 }
76 }
77 } else {
78     console.log("Unhandled stripe event:", event.type);
79 }
80
81 return res.json({ received: true });
82 } catch (err) {
83     console.error("Webhook processing error:", err);
84     return res.status(500).send();
85 }
86 };

```

server\middleware\auth.js

```

1 | import jwt from "jsonwebtoken";

```

```

2 import User from "../models/User.js";
3 import dotenv from 'dotenv';
4 dotenv.config()
5 export const verifyUser = async (req, res, next) => {
6   try {
7     let token = req.header("Authorization");
8     if (!token)
9       return res.status(401).json({ success: false, message: "Access Denied" });
10
11     if (token.startsWith("Bearer ")) {
12       token = token.slice(7).trim();
13     }
14
15     const decoded = jwt.verify(token, process.env.SECRET_KEY);
16     const user = await User.findById(decoded.id).select("-password");
17     if (!user) {
18       return res
19         .status(404)
20         .json({ success: false, message: "User not found" });
21     }
22
23     req.user = user;
24     next();
25   } catch (error) {
26     if (error.name === "TokenExpiredError") {
27       return res.status(401).json({ success: false, message: "Token expired" });
28     }
29     if (error.name === "JsonWebTokenError") {
30       return res.status(401).json({ success: false, message: "Invalid token" });
31     }
32     return res
33       .status(401)
34       .json({ success: false, message: "Authentication failed" });
35   }
36 };
37
38 export const authorize = (...roles) => {
39   return (req, res, next) => {
40     if (!req.user) {
41       return res.status(401).json({
42         success: false,
43         message: "Access denied",

```

```

44     });
45     }
46     if (!roles.includes(req.user.role)) {
47         return res.status(403).json({
48             success: false,
49             message: `User role ${req.user.role} is not authorized`,
50         });
51     }
52     next();
53 };
54 };
55

```

server\middleware\upload.js

```

1  import multer from "multer";
2  import { v2 as cloudinary } from "cloudinary";
3  import { CloudinaryStorage } from "multer-storage-cloudinary";
4
5  cloudinary.config({
6      cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
7      api_key: process.env.CLOUDINARY_API_KEY,
8      api_secret: process.env.CLOUDINARY_API_SECRET,
9  });
10
11  const storage = new CloudinaryStorage({
12      cloudinary,
13      params: {
14          folder: "ecommerce",
15          allowed_formats: ["jpg", "jpeg", "png", "webp"],
16          transformation: [{ width: 800, height: 800, crop: "limit" }],
17      },
18  });
19
20  const upload = multer({
21      storage,
22      limits: {
23          fileSize: 5 * 1024 * 1024,
24      },
25      fileFilter: (req, file, cb) => {
26          if (file.mimetype.startsWith("image/")) {
27              cb(null, true);

```

```
28     } else {
29         cb(new Error("Only image files are allowed"), false);
30     }
31 },
32 });
33
34 export default upload;
35
```

server\models\Cart.js

```
1 // import mongoose from "mongoose";
2
3 // const cartSchema = new mongoose.Schema(
4 //   {
5 //     user: {
6 //       type: mongoose.Schema.Types.ObjectId,
7 //       ref: "User",
8 //       required: true,
9 //     },
10 //     items: [
11 //       {
12 //         product: {
13 //           type: mongoose.Schema.Types.ObjectId,
14 //           ref: "Product",
15 //           required: true,
16 //         },
17 //         quantity: {
18 //           type: Number,
19 //           required: true,
20 //           min: 1,
21 //           default: 1,
22 //         },
23 //         price: {
24 //           type: Number,
25 //           required: true,
26 //         },
27 //       },
28 //     ],
29 //     totalPrice: {
30 //       type: Number,
31 //       required: true,
```

```

32 //      default: 0,
33 //    },
34 //  },
35 //  { timestamps: true }
36 // );
37
38 // export default mongoose.model("Cart", cartSchema);
39
40
41 // models/Cart.js
42 import mongoose from "mongoose";
43
44 const cartItemSchema = new mongoose.Schema({
45   product: {
46     type: mongoose.Schema.Types.ObjectId,
47     ref: "Product",
48     required: true
49   },
50   quantity: {
51     type: Number,
52     required: true,
53     min: 1,
54     default: 1
55   }
56 }, { _id: true }); // Make sure _id is enabled for cart items
57
58 const cartSchema = new mongoose.Schema({
59   user: {
60     type: mongoose.Schema.Types.ObjectId,
61     ref: "User",
62     required: true,
63     unique: true
64   },
65   items: [cartItemSchema]
66 }, { timestamps: true });
67
68 export default mongoose.model("Cart", cartSchema);

```

server\models\Category.js

```

1 import mongoose from "mongoose";
2

```

```

3  const categorySchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: true,
8        unique: true,
9        trim: true,
10     },
11     description: {
12       type: String,
13       default: "",
14     },
15   },
16   { timestamps: true }
17 );
18
19 const Category = mongoose.model("Category", categorySchema);
20 export default Category;
21

```

server\models\Order.js

```

1  import mongoose from "mongoose";
2
3  const orderSchema = new mongoose.Schema(
4    {
5      user: {
6        type: mongoose.Schema.Types.ObjectId,
7        ref: "User",
8        required: true,
9      },
10     orderItems: [
11       {
12         product: {
13           type: mongoose.Schema.Types.ObjectId,
14           ref: "Product",
15           required: true,
16         },
17         quantity: {
18           type: Number,
19           required: true,
20           min: 1,

```

```
21     },
22     price: {
23         type: Number,
24         required: true,
25     },
26 },
27 ],
28 shippingAddress: {
29     address: { type: String, required: true },
30     city: { type: String, required: true },
31     postalCode: { type: String },
32     country: { type: String, required: true },
33 },
34 totalPrice: {
35     type: Number,
36     required: true,
37     default: 0,
38 },
39 paymentMethod: {
40     type: String,
41     enum: ["Cash on Delivery", "Card", "PayPal"],
42     default: "Card",
43 },
44 paymentResult: {
45     id: String,
46     status: String,
47     update_time: String,
48     email_address: String,
49 },
50 isPaid: {
51     type: Boolean,
52     default: false,
53 },
54 paidAt: Date,
55 status: {
56     type: String,
57     enum: ["Pending", "Processing", "Shipped", "Delivered", "Cancelled"],
58     default: "Pending",
59 },
60 },
61 { timestamps: true }
62 );
```

```
63
64 export default mongoose.model("Order", orderSchema);
65
```

server\models\Product.js

```
1 import mongoose from 'mongoose';
2
3
4 const reviewSchema = new mongoose.Schema(
5   {
6     user: {
7       type: mongoose.Schema.Types.ObjectId,
8       ref: "User",
9       required: true,
10    },
11    userName: {
12      type: String,
13      required: true,
14    },
15    rating: {
16      type: Number,
17      required: true,
18      min: 1,
19      max: 5,
20    },
21    comment: {
22      type: String,
23      required: true,
24      trim: true,
25    },
26  },
27  { timestamps: true }
28 );
29
30 const productSchema = new mongoose.Schema({
31   name: {
32     type: String,
33     required: true,
34   },
35   description: {
36     type: String,
```

```
37   required: true,
38 },
39 price: {
40   type: Number,
41   required: true,
42 },
43 stock: {
44   type: Number,
45   required: true,
46 },
47 image: [
48   {
49     url: String,
50     filename: String,
51   },
52 ],
53 category: {
54   type: mongoose.Schema.Types.ObjectId,
55   ref: "Category",
56   required: true,
57 },
58 reviews: [reviewSchema],
59 rating: {
60   type: Number,
61   default: 0,
62   min: 0,
63   max: 5,
64 },
65 numReviews: {
66   type: Number,
67   default: 0,
68 },
69 });
70 productSchema.pre("save", function (next) {
71   if (this.reviews.length > 0) {
72     this.rating =
73       this.reviews.reduce((acc, item) => item.rating + acc, 0) /
74       this.reviews.length;
75     this.numReviews = this.reviews.length;
76   } else {
77     this.rating = 0;
78     this.numReviews = 0;
```

```
79 }
80   next();
81 });
82 const Product = mongoose.model("Product",productSchema);
83 export default Product;
84
```

server\models\Review.js

```
1 import mongoose from "mongoose";
2
3 const reviewSchema = new mongoose.Schema(
4   {
5     userId: {
6       type: mongoose.Schema.Types.ObjectId,
7       ref: "User",
8       required: true,
9     },
10    productId: {
11      type: mongoose.Schema.Types.ObjectId,
12      ref: "Product",
13      required: true,
14    },
15    rating: { type: Number, required: true, min: 1, max: 5 },
16    comment: { type: String },
17    images: [{ type: String }],
18    isVerifiedPurchase: { type: Boolean, default: false },
19  },
20  {
21    timestamps: true,
22  }
23 );
24
25 const Review = mongoose.model("Review", reviewSchema);
26 export default Review;
27
```

server\models\User.js

```
1 import mongoose from "mongoose";
2 const userSchema = new mongoose.Schema(
3   {
4     name: {
```

```

5     type: String,
6     required: true,
7 },
8
9 email: {
10    type: String,
11    required: true,
12    unique: true,
13 },
14 password: {
15    type: String,
16    required: true,
17    select: false,
18    min: 6,
19 },
20 role: {
21    type: String,
22    enum: ["user", "admin"],
23    default: "user",
24 },
25 },
26 { timestamps: true }
27 );
28
29 const User = mongoose.model("User", userSchema);
30 export default User;
31

```

server\package.json

```

1  {
2    "name": "server",
3    "version": "1.0.0",
4    "description": "",
5    "main": "server.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node server.js"
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",

```

```

13   "type": "module",
14   "dependencies": {
15     "bcrypt": "^6.0.0",
16     "body-parser": "^2.2.0",
17     "cloudinary": "^2.7.0",
18     "cors": "^2.8.5",
19     "dotenv": "^17.2.3",
20     "express": "^5.1.0",
21     "jsonwebtoken": "^9.0.2",
22     "mongoose": "^8.19.1",
23     "multer": "^2.0.2",
24     "multer-storage-cloudinary": "^4.0.0",
25     "nodemon": "^3.1.10",
26     "stripe": "^19.1.0"
27   }
28 }
29

```

server\routes\auth.js

```

1  import express from 'express';
2  import { verifyUser, authorize } from '../middleware/auth.js';
3  import { login, logout, register, getMe } from '../controllers/auth.js';
4  const router = express.Router();
5
6  router.post('/register', register);
7  router.post('/login', login);
8  router.get("/me", verifyUser, getMe)
9  router.post('/logout', verifyUser, logout);
10 export default router;

```

server\routes\cartRoutes.js

```

1  import express from "express";
2  import {
3    getCart,
4    addToCart,
5    updateCartItem,
6    removeFromCart,
7    clearCart,
8  } from "../controllers/cartController.js";
9  import { verifyUser } from "../middleware/auth.js"; // use verifyUser
10

```

```
11 const router = express.Router();
12
13 // Ensure authentication for all cart endpoints
14 router.use(verifyUser);
15
16 router.get("/", getCart);
17 router.post("/", addToCart);
18 router.put("/:itemId", updateCartItem);
19 router.delete("/:itemId", removeFromCart);
20 router.delete("/", clearCart);
21
22 export default router;
23
```

server\routes\categoryRoutes.js

```
1 import express from "express";
2 import {
3   createCategory,
4   deleteCategory,
5   getAllCategories,
6   updateCategory,
7 } from "../controllers/categoryController.js";
8 import { authorize, verifyUser } from "../middleware/auth.js";
9
10 const router = express.Router();
11
12 router.get("/", getAllCategories);
13 router.post("/", verifyUser, authorize("admin"), createCategory);
14 router.put("/:id", verifyUser, authorize("admin"), updateCategory);
15 router.delete("/:id", verifyUser, authorize("admin"), deleteCategory);
16
17 export default router;
18
```

server\routes\orderRoutes.js

```
1 import express from "express";
2 import {
3   createOrder,
4   getUserOrders,
5   updateOrderStatus,
6   deleteOrder,

```

```

7 } from "../controllers/orderController.js";
8 import { markOrderAsPaid } from "../controllers/paymentController.js";
9 import { authorize, verifyUser } from "../middleware/auth.js";
10
11 const router = express.Router();
12
13 router.post("/", verifyUser, createOrder);
14 router.get("/my-orders", verifyUser, getUserOrders);
15
16
17 router.put("/:id/status", verifyUser, authorize("admin"), updateOrderStatus);
18 router.delete("/:id", verifyUser, authorize("admin"), deleteOrder);
19
20 export default router;
21

```

server\routes\paymentRoutes.js

```

1 import express from "express";
2 import { createPaymentIntent, markOrderAsPaid } from "../controllers/paymentController.js";
3 import { authorize, verifyUser } from "../middleware/auth.js";
4
5 const router = express.Router();
6
7 router.post("/create-payment-intent", verifyUser, authorize("admin"), createPaymentIntent);
8
9 router.put("/order/:id/mark-paid", verifyUser, authorize("admin"), markOrderAsPaid);
10
11 export default router;
12

```

server\routes\productRoutes.js

```

1 import express from 'express'
2 import { getAllProducts, createProduct, getProductById, deleteProduct, updateProduct } from '../controllers/productController.js';
3 import {
4   createReview,
5   deleteReview,
6   getProductReviews,
7 } from "../controllers/reviewController.js";
8 import { verifyUser, authorize } from '../middleware/auth.js';
9 import upload from "../middleware/upload.js";
10

```

```
11 const router = express.Router();
12
13 router.get('/', getAllProducts);
14 router.get('/:id', getProductById);
15
16 router.post('/', verifyUser, authorize('admin'), upload.array('image', 5), createProduct);
17 router.put('/:id', verifyUser, authorize('admin'),upload.array('image',5), updateProduct);
18 router.delete('/:id', verifyUser, authorize('admin'), deleteProduct);
19
20 router.post("/:id/reviews", createReview);
21 router.get("/:id/reviews", getProductReviews);
22
23
24 router.delete("/:id/reviews/:reviewId", authorize("admin"), deleteReview);
25
26 export default router;
```

server\routes\rating.js

```
1 // routes/rating.js
2 import express from "express";
3 import {
4   addReview,
5   getProductReviews,
6   updateReview,
7   deleteReview,
8 } from "../controllers/ratingController.js";
9 import { verifyUser } from "../middleware/auth.js";
10
11 const router = express.Router();
12
13 // Add review
14 router.post("/products/:productId/reviews", verifyUser, addReview);
15
16 // Get product reviews
17 router.get("/products/:productId/reviews", getProductReviews);
18
19 // Update review
20 router.put("/products/:productId/reviews/:reviewId", verifyUser, updateReview);
21
22 // Delete review
23 router.delete("/products/:productId/reviews/:reviewId", verifyUser, deleteReview);
```

```
24
25 export default router;
26
```

server\routes\webhookRoutes.js

```
1 import express from "express";
2 const router = express.Router();
3
4 router.post(
5   "/",
6   express.raw({ type: "application/json" }),
7   (req, res) => {
8     req.rawBody = req.body;
9     req.body = {};
10    import("../controllers/webhookController.js").then(module => {
11      module.stripeWebhook(req, res);
12    });
13  }
14 );
15
16 export default router;
```

server\server.js

```
1 // server.js
2 import express from "express";
3 import dotenv from "dotenv";
4 import mongoose from "mongoose";
5 import cors from "cors";
6 // import cookieParser from "cookie-parser";
7 import authRoutes from "./routes/auth.js";
8 import productRoutes from "./routes/productRoutes.js";
9 import cartRoutes from "./routes/cartRoutes.js";
10 import orderRoutes from "./routes/orderRoutes.js";
11 import ratingRoutes from "./routes/rating.js";
12 // import adminRoutes from "./routes/admin.js";
13 import { verifyUser } from "./middleware/auth.js";
14
15 dotenv.config();
16
17 const app = express();
18
```

```
19 // Security / parse
20 app.use(express.json({ limit: "10mb" }));
21 app.use(express.urlencoded({ extended: true }));
22 // app.use(cookieParser());
23
24 // CORS - allow your frontend origin
25 app.use(
26   cors({
27     origin: process.env.CLIENT_URL || "http://localhost:5173",
28     credentials: true,
29   })
30 );
31
32 // Connect DB
33 mongoose
34   .connect(process.env.MONGO_URI, {
35     useNewUrlParser: true,
36     useUnifiedTopology: true,
37   })
38   .then(() => console.log("MongoDB connected"))
39   .catch((err) => {
40     console.error("MongoDB connection error:", err);
41     process.exit(1);
42   });
43
44 // Routes (keep names as in your project)
45 app.use("/api/auth", authRoutes);
46 app.use("/api/products", productRoutes);
47 app.use("/api/cart", cartRoutes);
48 app.use("/api/orders", orderRoutes);
49 app.use("/api", ratingRoutes);
50 // app.use("/api/admin", adminRoutes);
51
52 // health
53 app.get("/api/health", (req, res) => res.json({ ok: true }));
54
55 // Generic error handler
56 app.use((err, req, res, next) => {
57   console.error(err.stack);
58   res.status(err.status || 500).json({ success: false, message: err.message });
59 });
60
```

```
61 const port = process.env.PORT || 5000;  
62 app.listen(port, () => console.log(`Server running on port ${port}`));  
63
```